A
Practical Activity Report

Submitted for

DATABASE ENGINEERING-(UCS677)

END-Semester Lab Evaluation

JobHunt — Full Stack Job Portal System

Submitted to-
Course Faculty

BE Third Year

Submitted by-
Dron Garg (102303583)

Computer Science and Engineering Department
TIET, Patiala

January–May 2026

Contents

1 INTRODUCTION **3**
1.1 Project Overview . . . 3
1.2 Technology Stack . . . 3
1.3 Key Features . . . 3
1.4 Project Significance . . . 3

2 PROBLEM STATEMENT **4**
2.1 Current Market Challenges . . . 4
2.2 Job Seeker Challenges . . . 4
2.3 Recruiter Difficulties . . . 4
2.4 Platform Administration . . . 4
2.5 Proposed Solution . . . 4

3 SPECIFIC REQUIREMENTS **5**
3.1 Functional Requirements . . . 5
3.1.1 User Authentication and Authorisation . . . 5
3.1.2 Job Management . . . 5
3.1.3 Advanced Search and Filtering . . . 5
3.1.4 Job Bookmarking . . . 5
3.1.5 Application System . . . 5
3.1.6 Admin Dashboard . . . 5
3.2 Non-Functional Requirements . . . 6
3.2.1 Scalability . . . 6
3.2.2 Performance . . . 6
3.2.3 Security . . . 6
3.2.4 Reliability . . . 6
3.2.5 Maintainability . . . 6

4 DATABASE SCHEMA DESIGN **7**
4.1 Collections Overview . . . 7
4.2 Users Collection . . . 7
4.3 Jobs Collection . . . 8
4.4 Applications Collection . . . 8
4.5 Bookmarks Collection . . . 9

5 CONTEXT LEVEL DIAGRAM AND DATA FLOW DIAGRAM **10**
5.1 Context Level Diagram (Level 0) . . . 10
5.2 Data Flow Diagram (Level 1) . . . 10

6 SYSTEM SPECIFICATION **12**
6.1 Hardware Specifications . . . 12
6.1.1 Development Environment . . . 12
6.2 Software Specifications . . . 12
6.2.1 Runtime Environment . . . 12
6.2.2 Backend Dependencies . . . 12
6.2.3 Frontend Dependencies . . . 12
6.2.4 Development Tools . . . 13

7 TOOLS USED **14**
7.1 Backend Technologies . . . 14
7.1.1 MongoDB and Mongoose . . . 14
7.1.2 Express.js . . . 14
7.1.3 JSON Web Tokens (JWT) . . . 14
7.1.4 bcryptjs . . . 14
7.2 Frontend Technologies . . . 14
7.2.1 React.js with Vite . . . 14
7.2.2 React Router DOM . . . 14
7.2.3 Tailwind CSS . . . 14
7.2.4 Axios . . . 15
7.3 Testing Tools . . . 15
7.3.1 Jest and Supertest . . . 15
7.3.2 Postman Collection . . . 15

8 MONGODB CONCEPTS DEMONSTRATED **16**
8.1 Nested Arrays . . . 16
8.2 Indexing . . . 16
8.3 Update Operators . . . 16
8.4 Sharding Strategy . . . 16
8.5 Populate (Manual Joins) . . . 17

9 API REFERENCE **18**

10 SAMPLE SCREENSHOTS **19**

11 TESTING **25**
11.1 Automated Test Suite . . . 25
11.2 Manual Testing with Postman . . . 25
11.3 Database Verification with MongoDB Compass . . . 25

12 CONCLUSION **26**
12.1 Project Achievements . . . 26
12.2 MongoDB Concepts Covered . . . 26
12.3 Technical Accomplishments . . . 26
12.4 Future Enhancements . . . 26
12.5 Final Remarks . . . 27

1 INTRODUCTION

1.1 Project Overview

JobHunt is a comprehensive full-stack job portal system developed as part of the Database Engineering course project. The platform bridges the gap between job seekers and employers by providing a role-based web application backed by a MongoDB database. Unlike traditional frontend-only solutions, this project emphasises database design, indexing strategies, nested document structures, and real-time communication — all fundamental concepts of modern database-driven applications.

1.2 Technology Stack

The project is built on a two-tier architecture:

- **Backend**: Node.js with Express.js framework, MongoDB as the primary database via Mongoose ODM, JSON Web Tokens (JWT) for stateless authentication, and bcryptjs for password hashing.
- **Frontend**: React.js (Vite) with React Router DOM for client-side navigation, Tailwind CSS for styling, and Axios for HTTP communication.

1.3 Key Features

The platform provides a comprehensive set of features organised by user role:

- **Role-based access control** with three distinct roles: Admin, Applicant, and Recruiter, each with specific permissions and views.
- **Job management** including creation, editing, deletion, and advanced filtering by category, location, salary range, and full-text keyword search.
- **Application tracking** allowing applicants to apply, track status (In Progress, Shortlisted, Rejected), and withdraw applications with backend withdrawal counting.
- **Bookmarking system** for applicants to save and revisit job listings.
- **Admin dashboard** providing platform-wide statistics, user blacklisting, and full job moderation capabilities.

1.4 Project Significance

This project demonstrates the practical application of MongoDB features in a production-style environment. It showcases nested arrays, compound indexing, text search indexes, the `$inc` update operator, sharding strategy, and manual population (joins) across multiple collections — all within the context of a real-world use case that is immediately relatable and testable.

2 PROBLEM STATEMENT

2.1 Current Market Challenges

Online job portals are an essential part of the modern recruitment ecosystem. However, many existing platforms suffer from poor database design, lack of real-time capabilities, and rigid role models that do not accurately reflect the needs of applicants, recruiters, and platform administrators. This project addresses these challenges by designing a well-structured, scalable database schema from the ground up.

2.2 Job Seeker Challenges

Applicants face the following specific challenges that this project addresses:

- Inability to efficiently search and filter job listings across multiple criteria simultaneously.
- No mechanism to track the progress of submitted applications in real time.
- Lack of a persistent bookmarking system to save and review jobs of interest.

2.3 Recruiter Difficulties

Recruiters encounter their own set of challenges:

- Difficulty posting structured job listings with detailed skill requirements.
- No centralised view of all applicants for a given job posting with status management.
- Inability to communicate rejection or shortlisting decisions with contextual notes to applicants.

2.4 Platform Administration

Platform administrators need tools to:

- Monitor platform health through aggregate statistics across users, jobs, and applications.
- Moderate job listings to maintain content quality.
- Manage user accounts, including blacklisting repeat offenders who abuse the withdrawal system.

2.5 Proposed Solution

JobHunt addresses all of the above by implementing a MongoDB-backed REST API with JWT-secured endpoints and role-based middleware. The database schema is designed with MongoDB best practices including appropriate indexing, nested subdocument arrays, and text search capabilities — ensuring both correctness and performance at scale.

3 SPECIFIC REQUIREMENTS

3.1 Functional Requirements

3.1.1 User Authentication and Authorisation

The system supports secure user registration and login using email and password credentials. Passwords are hashed using bcryptjs before storage. JWT tokens are issued on login and verified by middleware on every protected route. The system enforces three distinct roles — Admin, Applicant, and Recruiter — with route-level guards preventing cross-role access. The admin account is pre-seeded with fixed credentials and cannot be self-registered.

3.1.2 Job Management

Recruiters can post jobs with the following fields: title, company name, location, salary (in INR), description, category (computer, analytics, design, electronics), tags array, and a nested requirements array containing skill name, proficiency level, and required flag. Recruiters can edit and delete their own jobs. Admins can edit or delete any job. The system supports full-text search across title, description, and tags using MongoDB text indexes.

3.1.3 Advanced Search and Filtering

Applicants and admins can filter jobs by category, location (regex match), salary range (`$gte`/`$lte`), and keyword (full-text search). Results can be sorted by newest, salary ascending, or salary descending using compound indexes.

3.1.4 Job Bookmarking

Applicants can bookmark jobs for later review. A unique compound index on `(applicantId, jobId)` prevents duplicate bookmarks. Bookmarks are displayed with full job detail via Mongoose populate.

3.1.5 Application System

Applicants can apply to active jobs by providing their name, LinkedIn profile (required), and GitHub profile (optional). A unique compound index prevents duplicate applications. Recruiters update application statuses (In Progress, Shortlisted, Rejected) with optional reason notes. Applicants can withdraw applications; each withdrawal increments a `withdrawalCount` field on the user document using MongoDB's `$inc` operator. Admins can view withdrawal counts and blacklist users who abuse the system.

3.1.6 Admin Dashboard

The admin dashboard provides platform statistics (total users, total jobs, total applications, blacklisted users) via aggregated queries. Admins can view and filter all users by role, toggle blacklist status on any applicant or recruiter, view all jobs including inactive ones, and delete or deactivate any job.

3.2 Non-Functional Requirements

3.2.1 Scalability

The database schema supports horizontal scaling via MongoDB sharding. The `jobs` collection is designed to be sharded on the `category` field to distribute read load across job type partitions.

3.2.2 Performance

All frequently queried fields are indexed. Compound indexes support combined filter and sort operations. Full-text indexes enable efficient keyword search without scanning entire collections.

3.2.3 Security

All passwords are bcrypt-hashed before persistence. JWT tokens expire after 7 days. Role-based middleware enforces access control on every protected route. Blacklisted users are blocked at the middleware level on every request, not just at login.

3.2.4 Reliability

Mongoose schema validation ensures data integrity at the application layer. Unique indexes enforce data constraints at the database layer (e.g., no duplicate applications, no duplicate bookmarks, no duplicate email addresses).

3.2.5 Maintainability

The project follows an MVC-style architecture separating models, controllers, routes, and middleware into distinct directories for clarity and ease of maintenance.

4 DATABASE SCHEMA DESIGN

4.1 Collections Overview

The JobHunt database (`jobhunt`) consists of four primary collections:

Collection	Documents	Purpose
`users`	All roles (admin, applicant, recruiter)	Identity and auth
`jobs`	Job postings	Core job data
`applications`	Applicant-job links	Application tracking
`bookmarks`	Applicant-job links	Saved jobs

4.2 Users Collection

All three roles share a single collection using a `role` discriminator field. Role-specific fields are `null` when not applicable.

```
{
  username:        String (required),
  email:           String (required, unique),
  password:        String (bcrypt hashed),
  role:            Enum ["admin", "applicant", "recruiter"],

  // Applicant-only
  linkedinProfile: String,
  githubProfile:   String,
  withdrawalCount: Number (default: 0),
  isBlacklisted:   Boolean (default: false),

  // Recruiter-only
  companyName:     String,
  companyLocation: String,

  timestamps: true
}
```

Listing 1: User Schema

Indexes: `email` (unique), `role`, `(isBlacklisted, role)` compound.

4.3 Jobs Collection

The Jobs collection demonstrates MongoDB's nested array capabilities through the `requirements` subdocument array and `tags` string array.

```
{
  recruiterId:  ObjectId (ref: User),
  title:        String,
  companyName:  String,
  location:     String,
  salary:       Number,
  description:  String,
  category:     Enum ["computer","analytics","design","electronics
     "],

  // Nested string array
  tags:         [String],

  // Nested subdocument array
  requirements: [{
    skill:    String,
    level:    Enum ["beginner","intermediate","expert"],
    required: Boolean
  }],

  isActive:     Boolean (default: true),
  timestamps:   true
}
```

Listing 2: Job Schema with Nested Arrays

Indexes: `category`, `salary`, `location`, `recruiterId`, `tags`, `isActive`, compound `(category, salary)`, text index on `(title, description, tags)`.

4.4 Applications Collection

```
{
  jobId:           ObjectId (ref: Job),
  applicantId:     ObjectId (ref: User),
  applicantName:   String,
  linkedinProfile: String,
  githubProfile:   String,
  status:          Enum ["in_progress","shortlisted","rejected"],
  statusReason:    String,
  isWithdrawn:     Boolean (default: false),
  withdrawnAt:     Date,
  timestamps:      true
}
```

Listing 3: Application Schema

Indexes: `applicantId`, `jobId`, `(jobId, status)` compound, `(applicantId, jobId)` unique compound (prevents duplicate applications).

4.5 Bookmarks Collection

```
{
  applicantId: ObjectId (ref: User),
  jobId:       ObjectId (ref: Job),
  timestamps:  true
}
```

Listing 4: Bookmark Schema

Indexes: `applicantId`, `(applicantId, jobId)` unique compound (prevents duplicate bookmarks).

5 CONTEXT LEVEL DIAGRAM AND DATA FLOW DIAGRAM

5.1 Context Level Diagram (Level 0)

The Context Level Diagram represents the JobHunt system as a single process interacting with three external entities:

- **Applicants**: Users who register, browse jobs, apply, bookmark, and manage their applications.
- **Recruiters**: Users who post and manage jobs, and update the status of received applications.
- **Administrators**: Pre-seeded superusers who moderate jobs and users across the platform.

All data flows through a REST API layer and are persisted to a MongoDB database running on localhost.

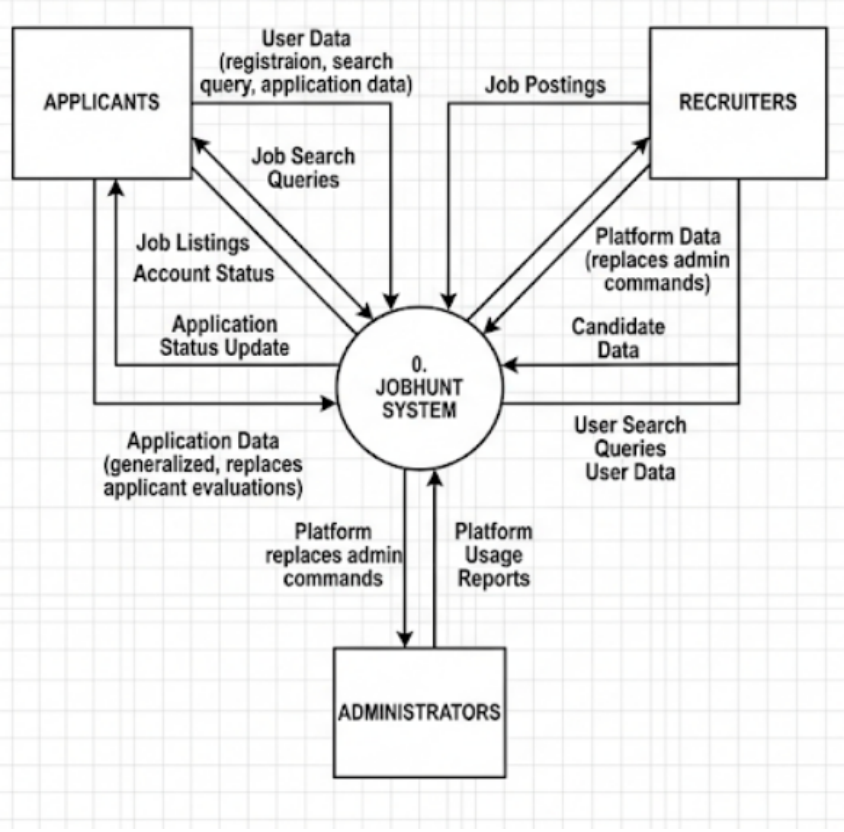

Figure 1: Context Diagram Level 0

5.2 Data Flow Diagram (Level 1)

The Level 1 DFD decomposes the JobHunt system into four major processes:

1. **Authentication Process**: Handles signup (with role selection), login (with JWT issuance), and logout. Reads and writes to the `users` collection. Middleware validates JWT on all protected routes and blocks blacklisted users.

2. **Job Management Process**: Handles job CRUD operations. Recruiters write to the `jobs` collection; applicants and admins read. Supports full-text search, category/location/salary filtering, and compound sort operations using MongoDB indexes.

3. **Application Process**: Manages the full application lifecycle. Applicants write to `applications`; recruiters update `status` and `statusReason`. Withdrawal uses `$inc` on the user document. Unique compound index enforces one application per applicant per job.

4. **Bookmarks Process**: Applicants read and write to the `bookmarks` collection. Uses populate to join with `jobs` for full job details on retrieval.

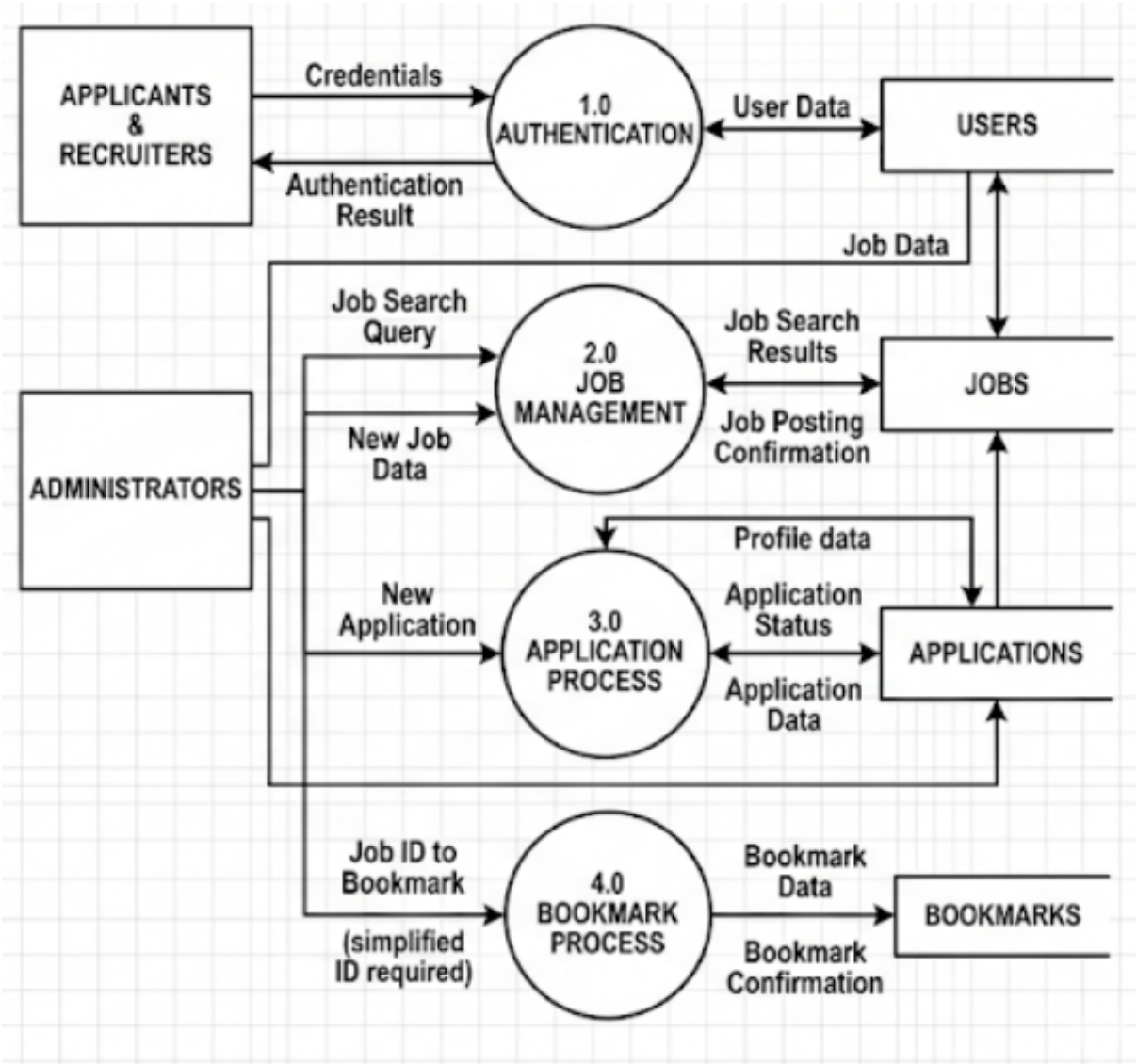

Figure 2: Data Flow Diagram Level 1

6 SYSTEM SPECIFICATION

6.1 Hardware Specifications

6.1.1 Development Environment

- **Processor**: Intel Core i5/i7 or AMD Ryzen 5/7 (or equivalent)
- **RAM**: 8 GB minimum; 16 GB recommended
- **Storage**: 256 GB SSD minimum
- **Display**: Full HD (1920x1080) resolution monitor

6.2 Software Specifications

6.2.1 Runtime Environment

- **Node.js**: v18+ (LTS)
- **MongoDB**: v6+ (Community Edition, local instance)
- **npm**: v9+

6.2.2 Backend Dependencies

- **express** 4.18.2 — Web framework and routing
- **mongoose** 7.0.0 — MongoDB ODM with schema validation
- **jsonwebtoken** 9.0.0 — JWT generation and verification
- **bcryptjs** 2.4.3 — Password hashing
- **cors** 2.8.5 — Cross-Origin Resource Sharing
- **dotenv** 16.0.3 — Environment variable management

6.2.3 Frontend Dependencies

- **react** 18.2.0 — Component-based UI library
- **react-router-dom** 6.20.0 — Client-side routing
- **axios** 1.6.0 — HTTP client with interceptors
- **tailwindcss** 3.3.6 — Utility-first CSS framework
- **vite** 5.0.0 — Frontend build tool and dev server

6.2.4 Development Tools

- **nodemon** — Auto-restart backend on file changes
- **jest + supertest** — Backend API testing
- **MongoDB Compass** — GUI for database inspection and query testing
- **Postman** — API testing with pre-built collection
- **Visual Studio Code** — Primary IDE

7 TOOLS USED

7.1 Backend Technologies

7.1.1 MongoDB and Mongoose

MongoDB is the primary data store for all application data. Mongoose provides schema definitions, validation, middleware hooks (pre-save for password hashing), and the populate API for joining related documents. The project uses multiple MongoDB features including text indexes, compound indexes, nested subdocument arrays, and the `$inc`, `$push`, and `$set` update operators.

7.1.2 Express.js

Express provides the HTTP server, routing, and middleware pipeline for the REST API. The project uses Express Router for modular route organisation and custom middleware for JWT authentication and role-based access control.

7.1.3 JSON Web Tokens (JWT)

JWT is used for stateless authentication. On login, a signed token containing the user's `_id` is returned to the client. On subsequent requests, the token is sent in the `Authorization` header and verified by the `protect` middleware before any protected resource is accessed.

7.1.4 bcryptjs

All user passwords are hashed using bcryptjs with a salt factor of 10 before being stored in MongoDB. Plain-text passwords are never persisted. The `matchPassword` instance method on the User model provides a clean API for login verification.

7.2 Frontend Technologies

7.2.1 React.js with Vite

React's component-based architecture enables reusable UI components (JobCard, FilterBar, StatusBadge, Navbar, ProtectedRoute) that are composed into full pages. Vite provides near-instant hot module replacement during development and optimised production builds.

7.2.2 React Router DOM

Provides client-side routing for a single-page application experience. Protected routes are wrapped in a custom `ProtectedRoute` component that checks authentication status and user role before rendering the target page.

7.2.3 Tailwind CSS

Tailwind's utility-first approach enables rapid UI development with a consistent design language. Custom component classes (btn-primary, card, input, badge) are defined in `index.css` using Tailwind's `@layer components` directive for reuse across the application.

7.2.4 Axios

Axios is configured with a base URL interceptor that automatically attaches the JWT token from localStorage to every outgoing request, eliminating the need for manual header management in individual API calls.

7.3 Testing Tools

7.3.1 Jest and Supertest

A comprehensive test suite covering all API endpoints was written using Jest and Supertest. Tests cover success paths, authentication failures, authorisation violations, duplicate data rejections, and 404 error handling.

7.3.2 Postman Collection

A Postman collection with pre-configured requests (including auto-saving tokens to collection variables via test scripts) is provided for manual API testing and demonstration. The collection covers all happy paths and expected failure cases (401, 403, 409 responses).

8 MONGODB CONCEPTS DEMONSTRATED

This section highlights how each MongoDB concept required by the course is implemented in the project.

8.1 Nested Arrays

Nested arrays are used in two collections:

- **jobs.tags**: A flat array of skill/keyword strings enabling array-based index queries (`db.jobs.create` `1}))`).
- **jobs.requirements**: An array of subdocuments, each with `skill`, `level`, and `required` fields.

8.2 Indexing

The project creates indexes across all collections:

- **Unique indexes**: `users.email`, `(applicantId, jobId)` in applications, `(applicantId, jobId)` in bookmarks.
- **Single-field indexes**: `category`, `salary`, `location`, `recruiterId`, `tags`, `role`, `isActive`.
- **Compound indexes**: `(category, salary)` for combined filter and sort; `(isBlacklisted, role)` for admin queries; `(jobId, status)` for recruiter response filtering.
- **Text index**: `(title, description, tags)` on the jobs collection enabling full-text keyword search using `$text: { $search: keyword }`.

8.3 Update Operators

- **$inc**: Used on `users.withdrawalCount` each time an applicant withdraws an application, atomically incrementing the counter.
- **$set**: Used when updating job fields and application status via `findByIdAndUpdate`.

8.4 Sharding Strategy

The `jobs` collection is designed for sharding on `{category: 1}`:

```
sh.enableSharding("jobhunt")
sh.shardCollection("jobhunt.jobs", { category: 1 })
```

Listing 5: Sharding Configuration

This distributes job documents across shards by category (computer, analytics, design, electronics), aligning with the primary query pattern of category-based filtering.

8.5 Populate (Manual Joins)

Mongoose's `populate()` is used throughout the project to join related documents:

- `applications` populate `jobId` with job title, company, location, salary, and category.
- `jobs` populate `recruiterId` with username, email, company name, and location.
- `bookmarks` populate `jobId` with full job details including `createdAt`.
- `applications/job/:jobId` populates `applicantId` with username, email, LinkedIn, GitHub, and `withdrawalCount`.

9 API REFERENCE

The backend exposes REST endpoints grouped by resource. All protected endpoints require a valid JWT in the `Authorization: Bearer <token>` header.

Method	Endpoint	Access	Description
POST	/api/auth/signup	Public	Register applicant/recruiter
POST	/api/auth/login	Public	Login, receive JWT
POST	/api/auth/logout	Any user	Logout
GET	/api/jobs	Public	Browse/filter/search jobs
GET	/api/jobs/:id	Public	Get single job detail
POST	/api/jobs	Recruiter	Post a new job
GET	/api/jobs/recruiter/my-jobs	Recruiter	Get own posted jobs
PUT	/api/jobs/:id	Recruiter/Admin	Edit job
DELETE	/api/jobs/:id	Recruiter/Admin	Delete job
POST	/api/applications/:jobId	Applicant	Apply to job
GET	/api/applications/my/applications	Applicant	My applications
DELETE	/api/applications/:id/withdraw	Applicant	Withdraw application
GET	/api/applications/job/:jobId	Recruiter/Admin	View responses
PUT	/api/applications/:id/status	Recruiter/Admin	Update status
POST	/api/bookmarks/:jobId	Applicant	Bookmark a job
GET	/api/bookmarks	Applicant	Get all bookmarks
DELETE	/api/bookmarks/:jobId	Applicant	Remove bookmark
GET	/api/admin/stats	Admin	Dashboard statistics
GET	/api/admin/users	Admin	All users
PUT	/api/admin/users/:id/blacklist	Admin	Toggle blacklist
GET	/api/admin/jobs	Admin	All jobs (incl. inactive)
PUT	/api/admin/jobs/:id	Admin	Edit any job
DELETE	/api/admin/jobs/:id	Admin	Delete any job

10 SAMPLE SCREENSHOTS

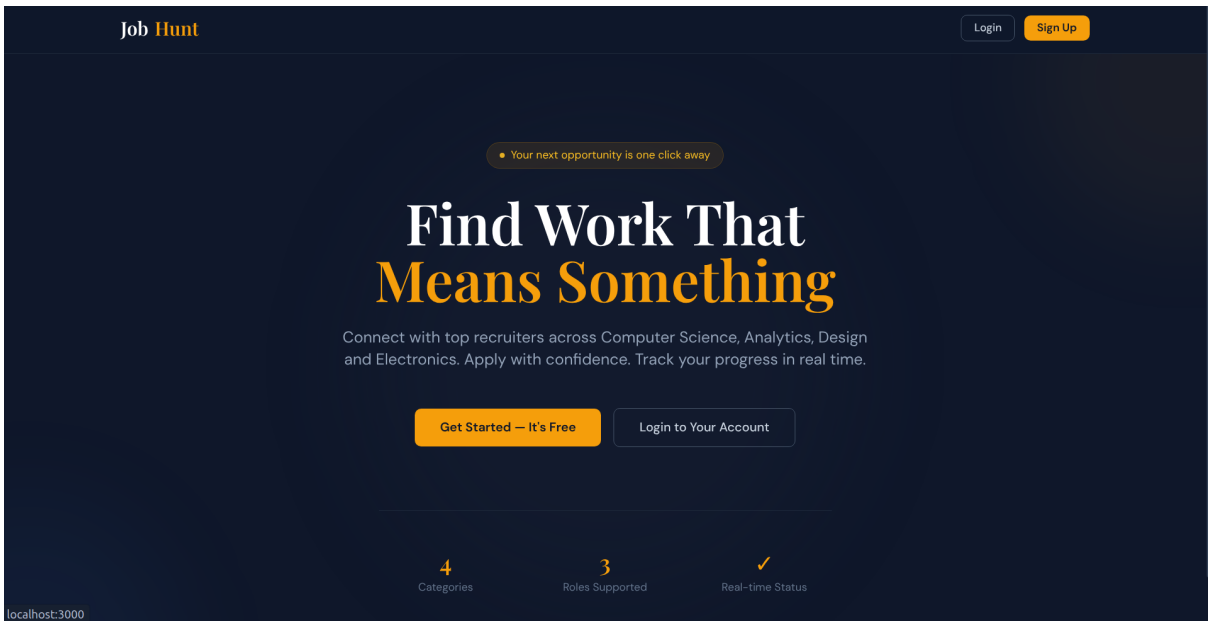

Figure 3: Home Page — Landing page with role-based CTA

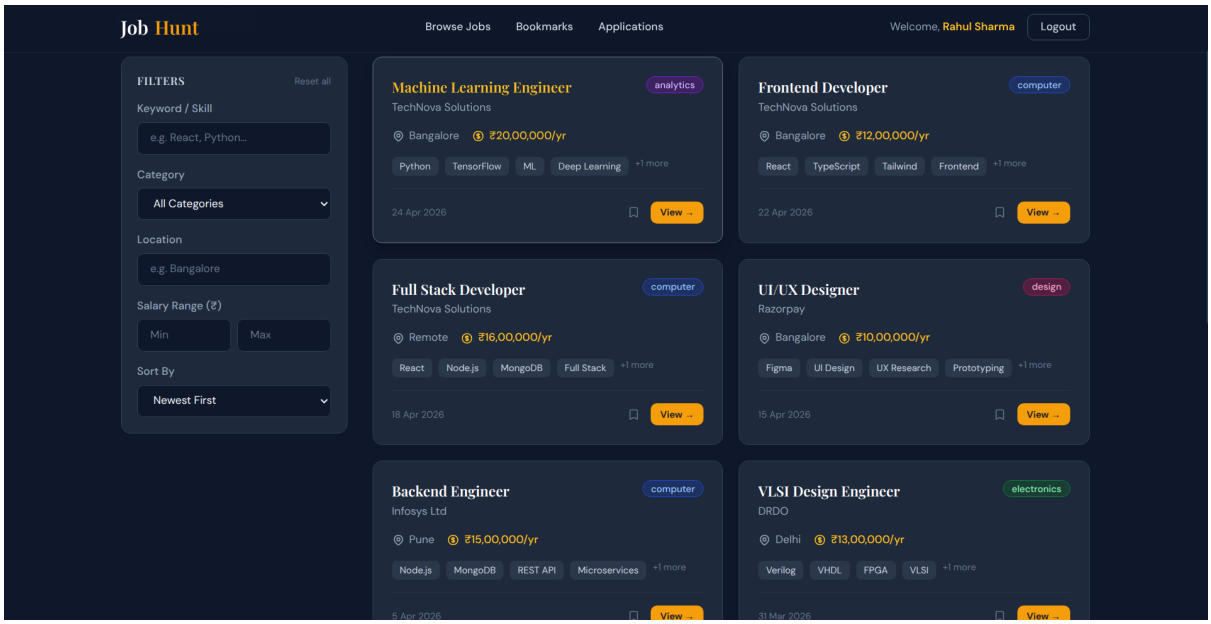

Figure 4: Jobs Page — Browse with filters (category, location, salary, keyword)

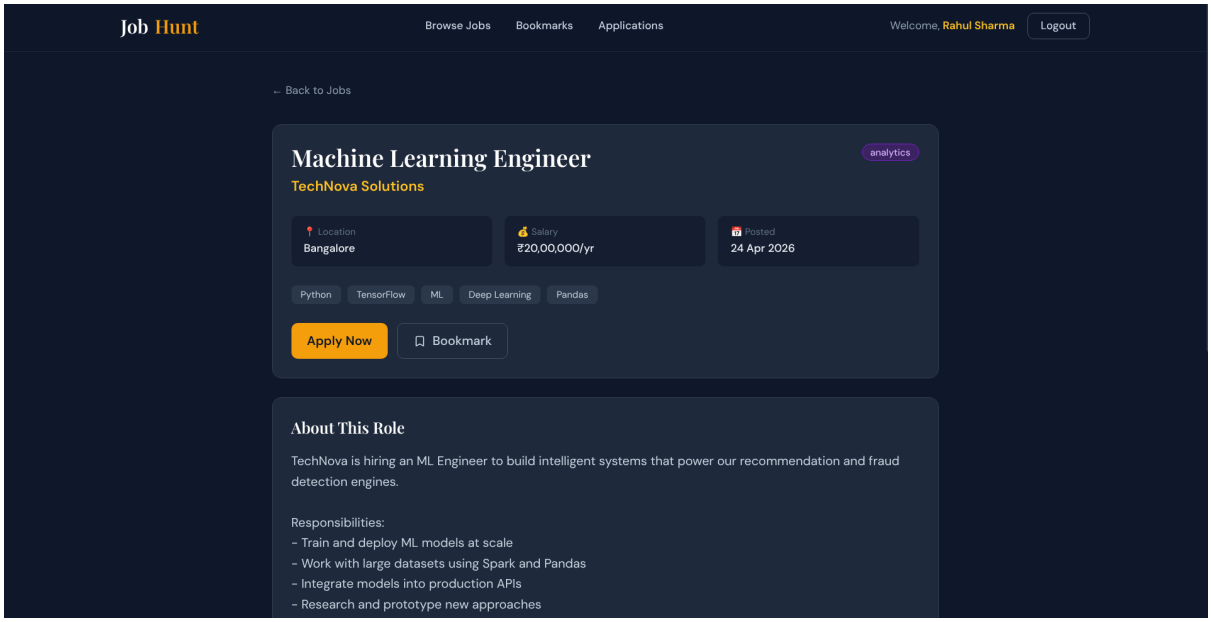

Figure 5: Job Detail Page — Full job info, requirements, company details, apply form

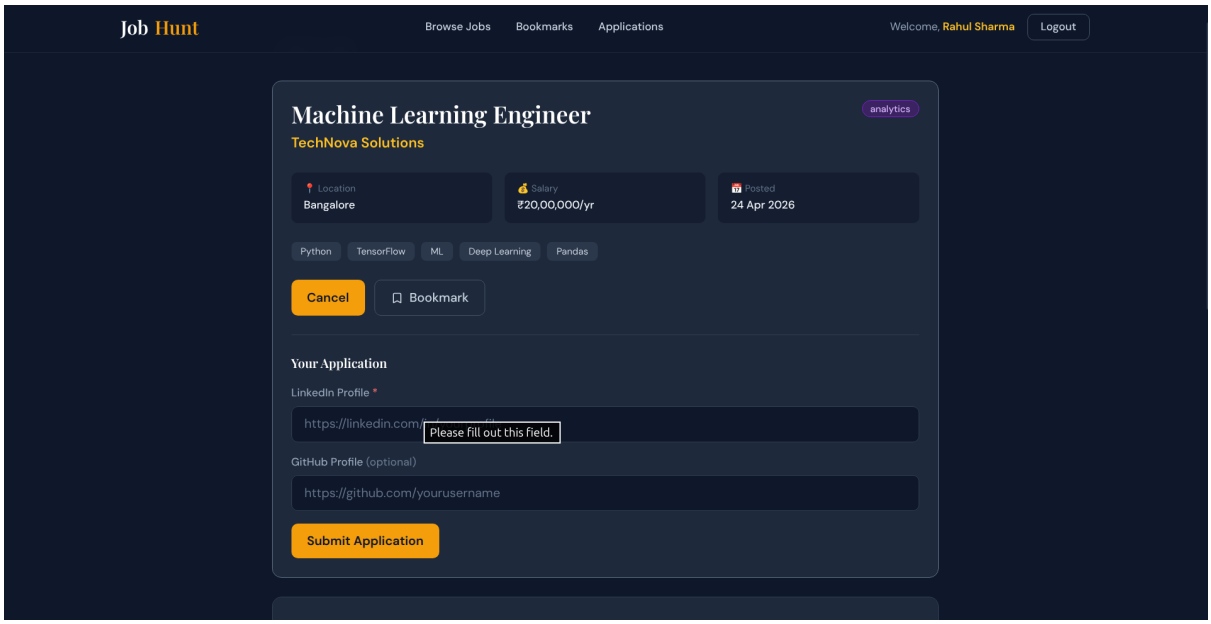

Figure 6: Apply Form — LinkedIn (required) and GitHub (optional) fields

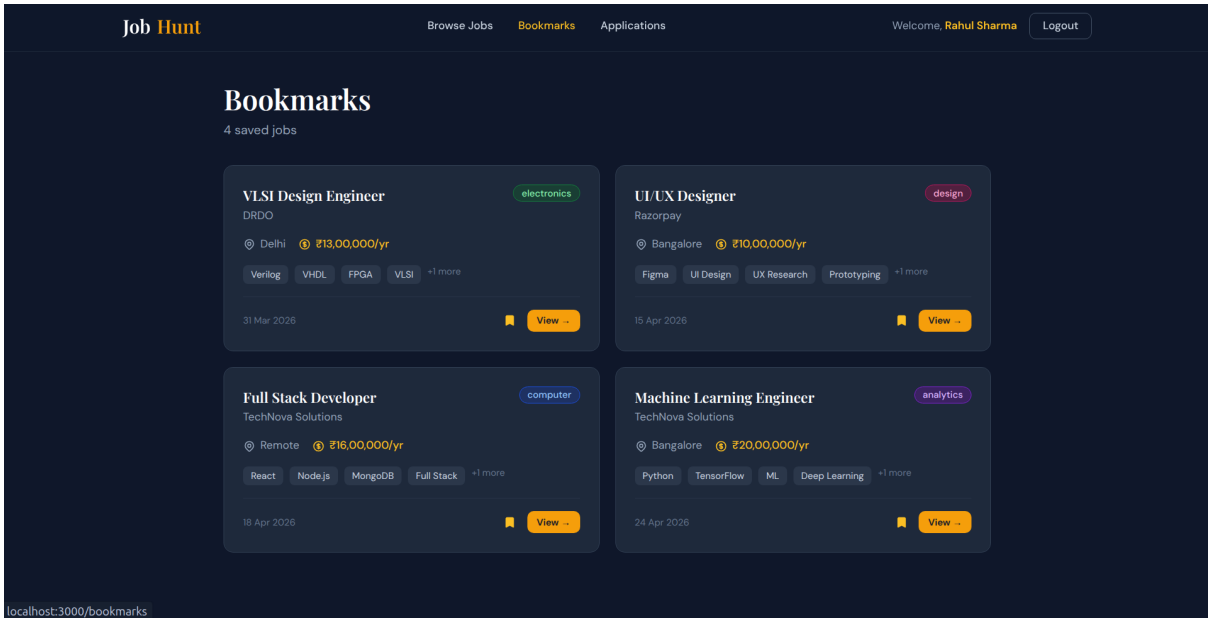

Figure 7: Bookmarks Page — Saved jobs with one-click removal

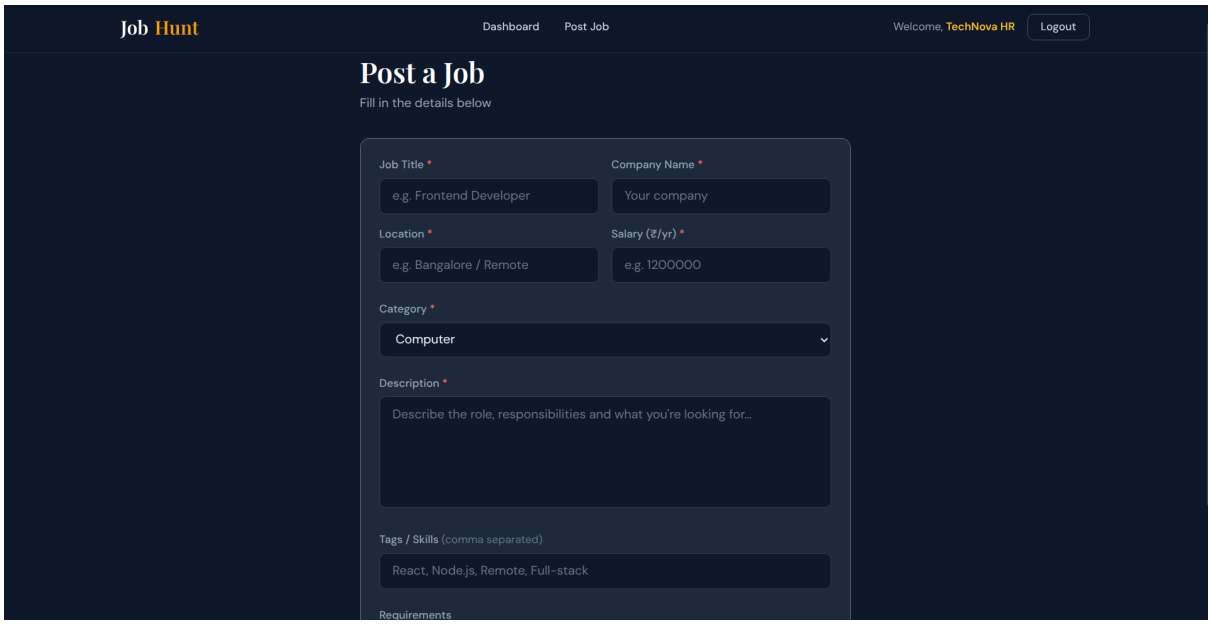

Figure 8: Post Job Page — Recruiter job creation with dynamic requirements builder

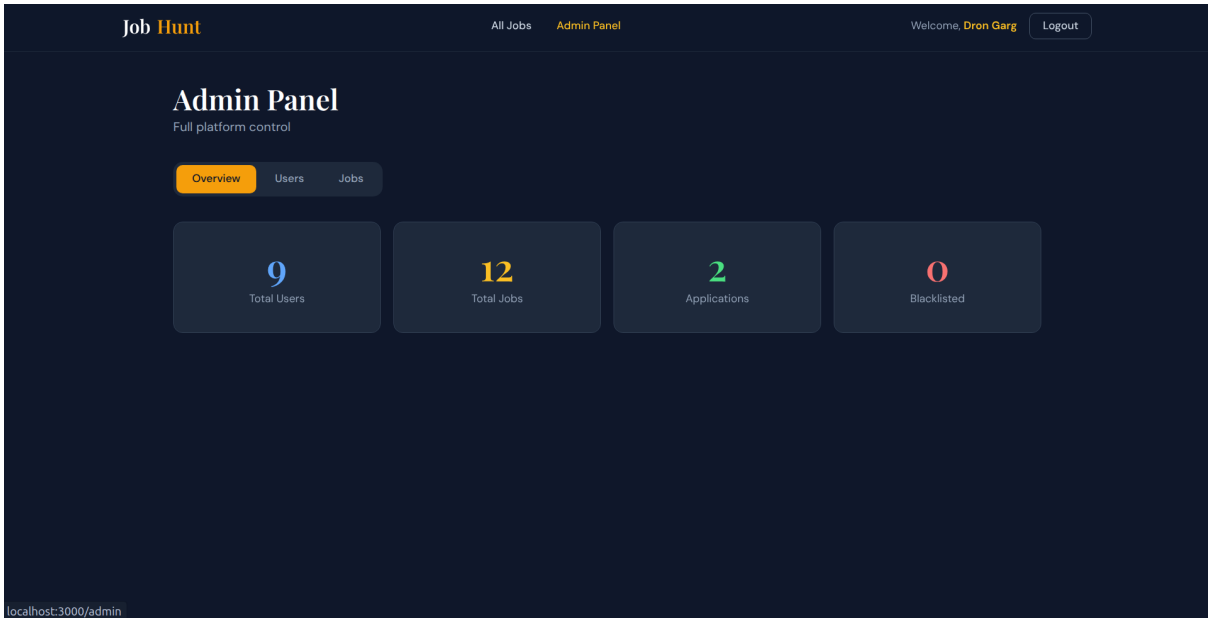

Figure 9: Admin Dashboard — Stats, user management, job moderation

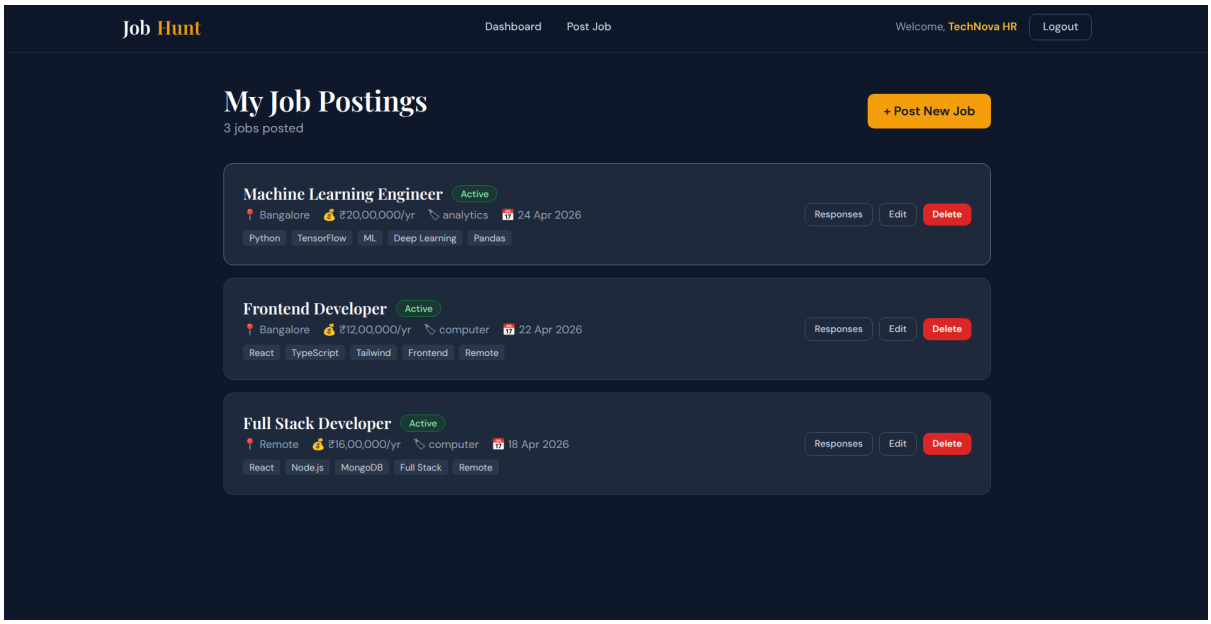

Figure 10: Recruiter Dashboard — Posted jobs with edit, delete, and responses

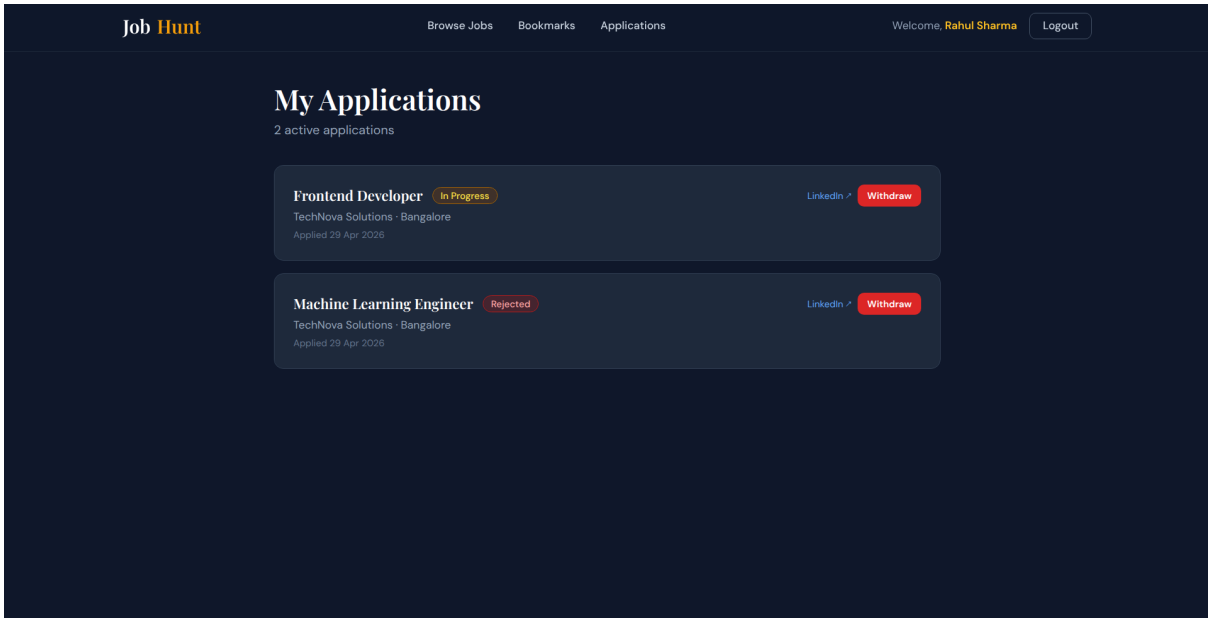

Figure 11: My Applications — Status tracking with company notes

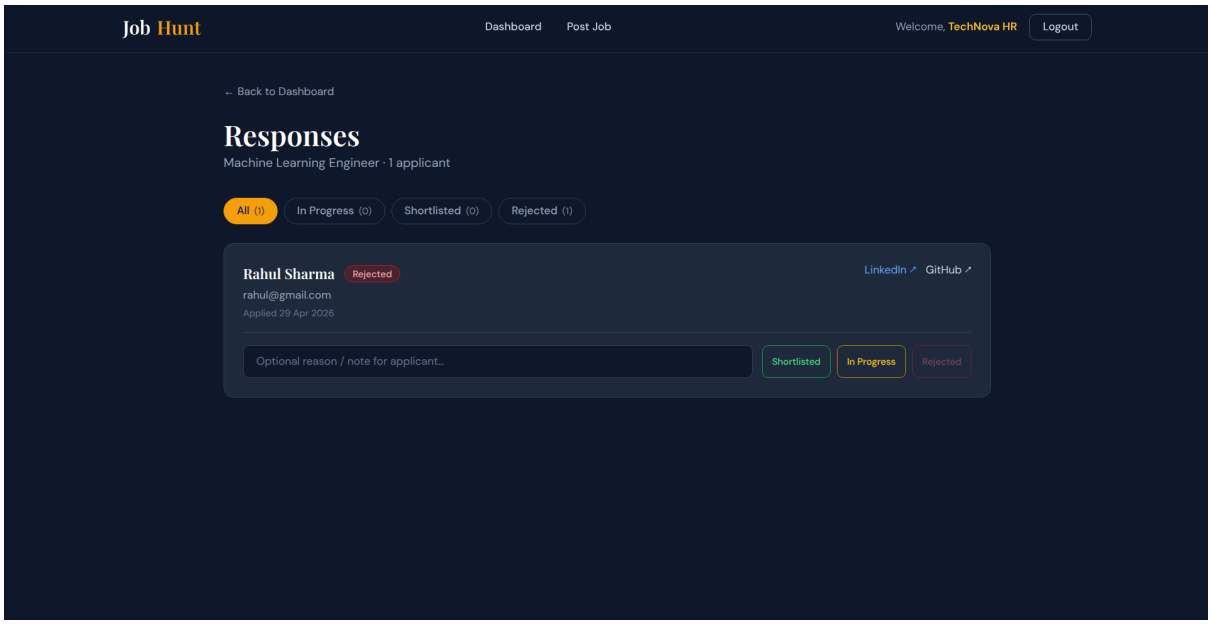

Figure 12: Response Page — View and manage applicant responses

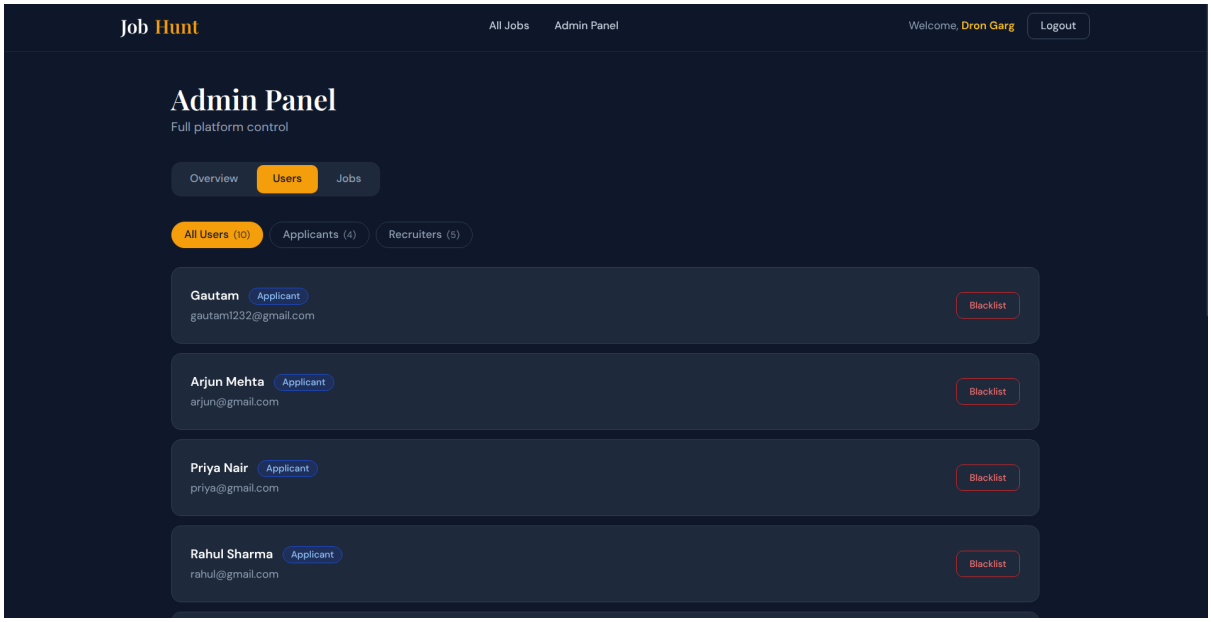

Figure 13: Blacklist Users — Admin control to restrict problematic users

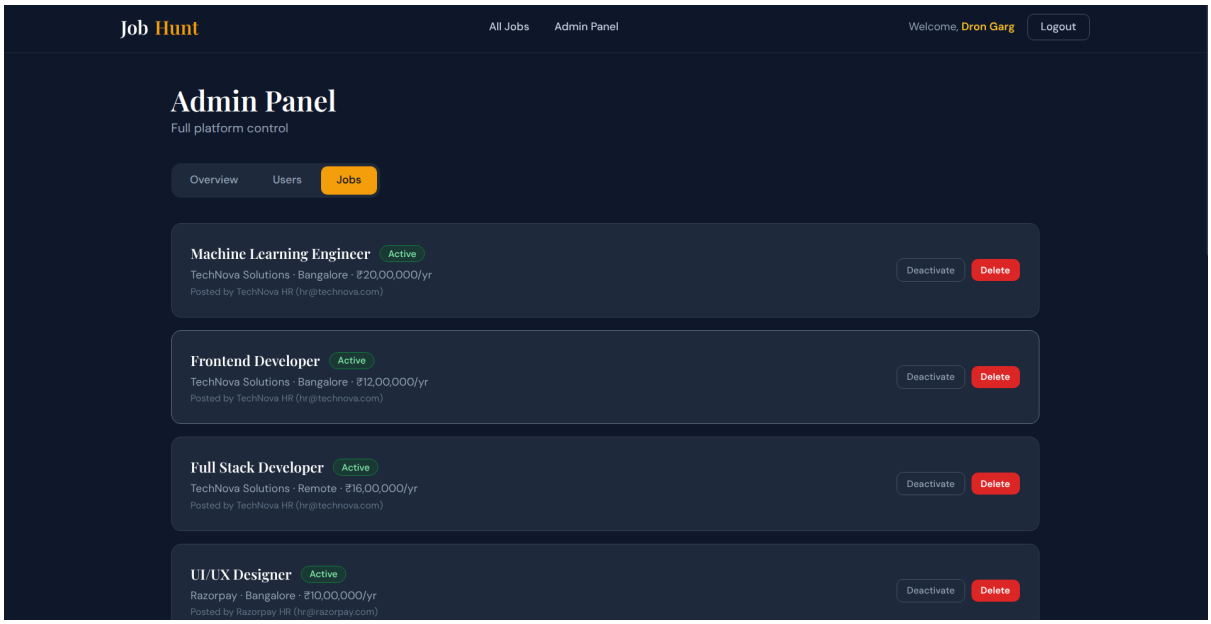

Figure 14: Job Handling — Admin tools for approving, editing, and removing jobs

11 TESTING

11.1 Automated Test Suite

A comprehensive automated test suite was written using Jest and Supertest covering all API endpoints. All tests pass successfully.

Test Suite	Coverage	Tests
Health	Server running	1
Authentication	Login, wrong password, non-existent user	3
Jobs	CRUD, filters, auth guard, missing fields	7
Applications	Apply, duplicate, withdraw, invalid job	5
Bookmarks	Add, duplicate, fetch, remove	4
Authorization	Role guards (403 enforcement)	2
Admin	Stats, users, non-admin blocked	3
Errors	404, invalid params, invalid ObjectId	3
Total		**28**

11.2 Manual Testing with Postman

A Postman collection with pre-configured requests is provided. Tokens and IDs are auto-saved between requests using test scripts, enabling the full test flow to be executed sequentially without manual intervention. The collection covers all happy paths and expected failure cases (401, 403, 409 responses).

11.3 Database Verification with MongoDB Compass

MongoDB Compass was used throughout development to verify:

- Correct document structure and field types in all collections
- Index creation and usage via the Explain Plan feature
- Nested array integrity for job requirements
- Unique index enforcement for duplicate prevention

12 CONCLUSION

12.1 Project Achievements

The JobHunt Full Stack Job Portal System successfully demonstrates the practical application of MongoDB database design principles within a production-style full-stack web application. Starting from a well-reasoned schema design and progressing through a fully tested REST API to a polished React frontend, the project covers the complete lifecycle of a database-driven application.

12.2 MongoDB Concepts Covered

Every major MongoDB concept required by the course is demonstrated in a real, functional context rather than as isolated examples:

- **Nested arrays** in job requirements
- **Text indexing** for full-text job search
- **Compound indexes** for combined filter and sort operations
- **Unique indexes** for data integrity enforcement
- **Update operators** (`$inc`, `$set`) for atomic updates
- **Sharding strategy** for horizontal scaling of the jobs collection
- **Population (manual joins)** across all related collections
- **Schema validation** and middleware hooks via Mongoose

12.3 Technical Accomplishments

Beyond the database layer, the project demonstrates proficiency in:

- Stateless JWT-based authentication with role-based access control middleware
- A fully automated test suite with 28 passing tests
- A clean, modern frontend built with React, Vite, and Tailwind CSS

12.4 Future Enhancements

Potential improvements for future iterations include:

- Company logo uploads using MongoDB GridFS
- Email notifications for application status changes
- Pagination and infinite scroll for large job datasets

- AI-powered job recommendations based on applicant profile
- Deployment to cloud infrastructure (Railway for backend, Vercel for frontend, MongoDB Atlas for database)

12.5 Final Remarks

This project reflects a strong understanding of database-driven application development, from schema design and indexing strategy to API security and frontend integration. The use of MongoDB as the primary data store enabled flexible document modelling that would have been significantly more complex in a relational database, particularly for features like nested job requirements. The project provides a solid, extensible foundation for a production-grade job portal platform.